

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности
Кафедра «Информационная безопасность компьютерных систем»

ЛАБОРАТОРНАЯ РАБОТА № 3

«РАБОТА С ТАЙМЕРАМИ-СЧЁТЧИКАМИ»

по дисциплине «Аппаратные средства вычислительной техники»

Выполнил
студент гр. 5131001/20003

<подпись>

Черникова В.М.

Преподаватель

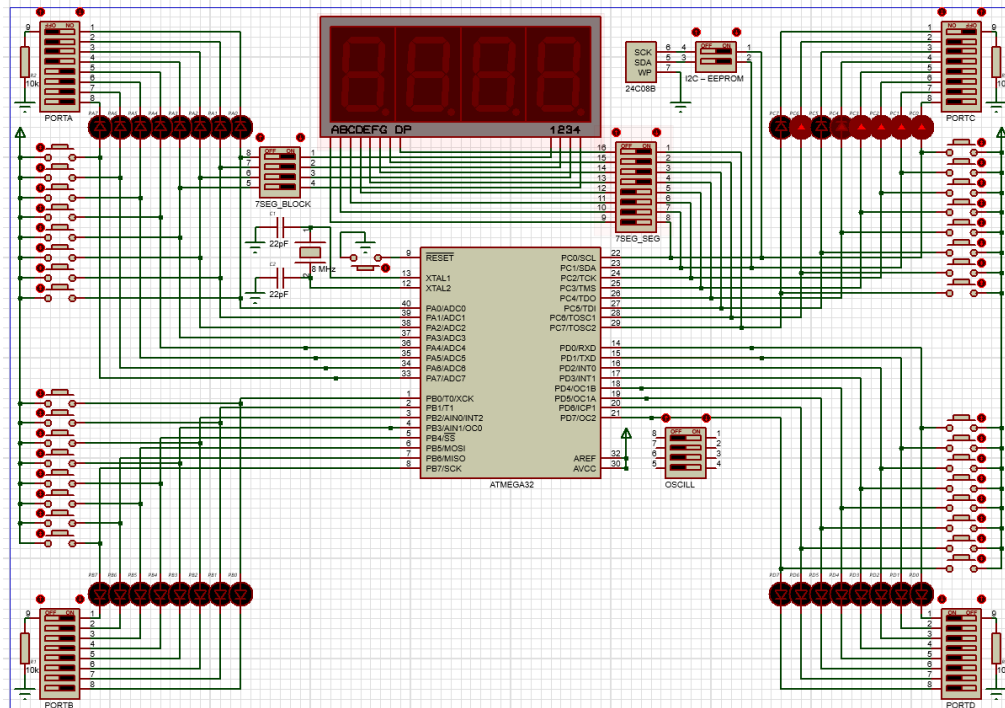
<подпись>

Семенов П.О.

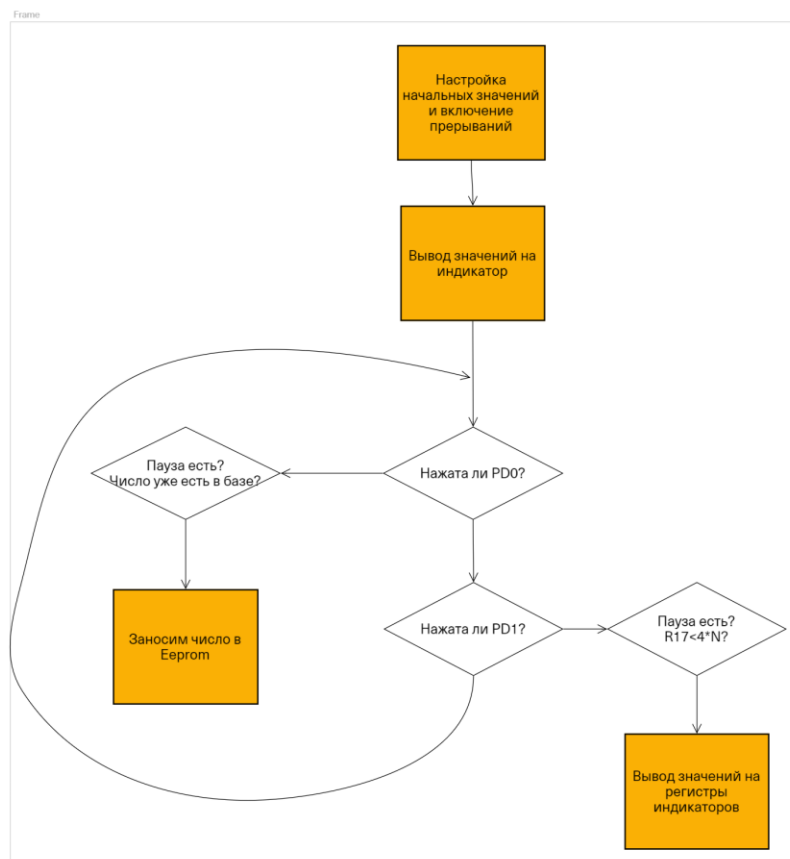
Санкт-Петербург
2024

1. Цель лабораторной работы — Получение практических навыков по работе с таймерами-счётчиками и применению механизма прерываний.

2. Схема установки (задействованные узлы отладочной платы).



3. Блок-схема алгоритма работы программы



4. Комментированный листинг программы на языке ассемблера.

```
.def TMP = R24
.def Pause=R16 ;1 - пауза, 0 - нет паузы
.def ZET = R19 ; буффер для EEPROM, r17 и r18 зарезервированы под адрес
.def N = R21
.def buf = R22
.def LED1 = R2
.def LED2 = R3
.def LED3 = R4
.def LED4 = R5

.org $000
JMP reset ; Указатель на начало программы
.org INT0addr
JMP EXT_INT0 ; Указатель на обработчик прерывания int0
.org INT1addr
JMP EXT_INT1 ; Указатель на обработчик прерывания int1
.org 0x00E
JMP EXT_TIMER1 ; Указатель на обработчик прерывания timer1 (сравнение)
.org 0x016
JMP EXT_TIMER0 ; Указатель на обработчик прерывания timer0 (переполнение)

.equ Zero = 0x3F
.equ One = 0x06
.equ Two = 0x5B
.equ Three = 0x4F
.equ Four = 0x66
.equ Five = 0x6D
.equ Six = 0x7D
.equ Seven = 0x07
.equ Eight = 0x7F
.equ Nine = 0x6F

.equ Zero_dot = 0xBF
.equ One_dot = 0x86
.equ Two_dot = 0xDB
.equ Three_dot = 0xCF
.equ Four_dot = 0xE6
.equ Five_dot = 0xED
.equ Six_dot = 0xFD
.equ Seven_dot = 0x87
.equ Eight_dot = 0xFF
.equ Nine_dot = 0xEF

;-----
; Функция кнопки D0 (r18:r17) - запись в память
VV_Y0:
SBIS PIND, 0
jmp e_p_eeep
CPI Pause, 1
BREQ e_p_eeep
BRNE p_eeep
p_eeep:

;CLR R18
LDI buf, 1
CPI N, 1
BRLO vse_ok
CLR R17
ADD R17, N
```

```

ADD R17, N
ADD R17, N
ADD R17, N
;----

SUBI R17, 4
CALL EEPROM_read
ADD R17, buf
CP ZET, LED1
BRNE vse_ok
CALL EEPROM_read
ADD R17, buf
CP ZET, LED2
BRNE vse_ok
CALL EEPROM_read
ADD R17, buf
CP ZET, LED3
BRNE vse_ok
CALL EEPROM_read
ADD R17, buf
CP ZET, LED4
BRNE vse_ok
jmp e_p_eep ; это же время уже есть в памяти

```

```

;----
vse_ok:
CLR R17
ADD R17, N
ADD R17, N
ADD R17, N
ADD R17, N
MOV ZET, LED1
CALL EEPROM_write

```

```

ADD R17, buf
MOV ZET, LED2
CALL EEPROM_write
ADD R17, buf
MOV ZET, LED3
CALL EEPROM_write
ADD R17, buf
MOV ZET, LED4
CALL EEPROM_write

```

```

ADD N, buf
;CPI N, 64
;BRLO e_p_eep
;проверка на N<64
;LDI N, 1
e_p_eep:

```

```

RET

```

```

VV_Y1:

```

```

SBIS PIND, 1

```

```

jmp e_s_eep

```

```

CPI Pause, 0

```

```

BREQ e_s_eep

```

```

CPI N, 0

```

```

BREQ e_s_eep

```

```

MOV TMP, N

```

```

ADD TMP, N

```

```

ADD TMP, N

```

```

    ADD TMP, N
CP R17, TMP
    BRLO s_eep ; sh lo
    CLR R17

```

```

s_eep:
    ;CLR R18
    LDI TMP, 1
    CALL EEPROM_read ; r18=0, r17=0,4,8... ,zet
    MOV LED1, ZET
    ADD R17, TMP
    CALL EEPROM_read
    MOV LED2, ZET
    ADD R17, TMP
    CALL EEPROM_read
    MOV LED3, ZET
    ADD R17, TMP
    CALL EEPROM_read
    MOV LED4, ZET
    ADD R17, TMP

```

```

e_s_eep:
    RET

```

```

;-----
; Начальная настройка

```

```

reset:
    ; Установка вершины стека в конец ОЗУ
    LDI TMP, HIGH(RAMEND) ; старшие разряды адреса
    OUT SPH, TMP
    LDI TMP, LOW(RAMEND) ; Младшие разряды адреса
    OUT SPL, TMP

    LDI TMP, 0x0F
    OUT MCUCR, TMP ; Настройка прерываний int0 и int1 на условие 0/1
    LDI TMP, 0xC0
    OUT GICR, TMP ; Разрешение прерываний int0 и int1
    OUT GIFR, TMP ; Предотвращение

```

```

;настройка таймера

```

```

    LDI TMP, 0b00000010
    OUT TCCR0, TMP
    LDI TMP, 0b10000000
    OUT TCCR1A, TMP
    LDI TMP, 0b00001100
    OUT TCCR1B, TMP
    CLR TMP
    OUT TCNT0, TMP
    LDI TMP, 0b00010001
    OUT TIMSK, TMP
    LDI TMP, 0x7A
    OUT OCR1AH, TMP
    LDI TMP, 0x12
    OUT OCR1AL, TMP

```

```

SEI

```

```

; настройка портов ввода-вывода

```

```

    LDI TMP, 0b00001111
    OUT DDRA, TMP ; Вывод
    SER TMP ; 0xFF
    OUT DDRC, TMP ; Вывод
    LDI TMP, 0b11110000

```

```
OUT DDRD, TMP ; Вывод + PD0 на ввод
```

```
;-----  
CLR TMP
```

```
; настройка исходных значений
```

```
LDI TMP, 1  
MOV R0, TMP  
LDI TMP, 1  
MOV R1, TMP  
LDI TMP, Zero  
MOV LED1, TMP  
MOV LED2, TMP  
MOV LED4, TMP  
LDI TMP, Zero_dot  
MOV LED3, TMP  
CLR N  
CLR R18  
CLR ZET  
LDI Pause, 0
```

```
; Основной цикл
```

```
loop:  
    OUT PORTA, R0  
    OUT PORTC, R1  
    nop  
  
    SBIC PIND, 0;если вызывается эта кнопка, то работает функция VV_Y0  
    CALL VV_Y0  
    SBIC PIND, 1;если вызывается эта кнопка, то работает функция VV_Y1  
    CALL VV_Y1  
  
    jmp loop  
  
;OUT PORTD, R31
```

```
EXT_INT0: ; Обработчик прерывания int0 (pd2)
```

```
PUSH TMP ; сохранение текущего значения TMP в стеке  
IN TMP, SREG  
PUSH TMP ; сохранение текущего значения SREG в стеке
```

```
;если пауза - делаем пуск
```

```
CPI Pause, 1  
BREQ to_play  
BRNE to_pause  
to_pause:  
    LDI Pause, 1  
    ;LDI TMP, 0b10000000  
    ;ADD LED3, TMP  
    LDI TMP, 0b00000001  
    OUT TIMSK, TMP  
    CLR R17 ; !!!!!!!!!!!  
    jmp end_pause_or_play  
to_play:  
    LDI Pause, 0  
    ;LDI TMP, 0b10000000  
    ;SUB LED3, TMP  
    LDI TMP, 0b00010001  
    OUT TIMSK, TMP  
  
end_pause_or_play:  
POP TMP
```


;изменение регистров LED - актуальных цифр!!!!

```
MOV TMP, LED1
CPI TMP, Zero
BREQ a1
CPI TMP, One
BREQ a2
CPI TMP, Two
BREQ a3
CPI TMP, Three
BREQ a4
CPI TMP, Four
BREQ a5
CPI TMP, Five
BREQ a6
CPI TMP, Six
BREQ a7
CPI TMP, Seven
BREQ a8
CPI TMP, Eight
BREQ a9
CPI TMP, Nine
BREQ a0
```

end_sec1:

```
    MOV LED1, TMP
    jmp end_tim1
```

a1:

```
LDI TMP, One
jmp end_sec1
```

a2:

```
LDI TMP, Two
jmp end_sec1
```

a3:

```
LDI TMP, Three
jmp end_sec1
```

a4:

```
LDI TMP, Four
jmp end_sec1
```

a5:

```
LDI TMP, Five
jmp end_sec1
```

a6:

```
LDI TMP, Six
jmp end_sec1
```

a7:

```
LDI TMP, Seven
jmp end_sec1
```

a8:

```
LDI TMP, Eight
jmp end_sec1
```

a9:

```
LDI TMP, Nine
jmp end_sec1
```

a0:

```
LDI TMP, Zero
jmp sec1
```

sec1:

```
    MOV LED1, TMP
```

```
    MOV TMP, LED2
CPI TMP, Zero
BREQ b1
CPI TMP, One
BREQ b2
```



```

CPI TMP, Two
BREQ b3
CPI TMP, Three
BREQ b4
CPI TMP, Four
BREQ b5
CPI TMP, Five
BREQ b0

```

```

end_sec2:
    MOV LED2, TMP
    jmp end_tim1
b1:
LDI TMP, One
jmp end_sec2
b2:
LDI TMP, Two
jmp end_sec2
b3:
LDI TMP, Three
jmp end_sec2
b4:
LDI TMP, Four
jmp end_sec2
b5:
LDI TMP, Five
jmp end_sec2
b0:
LDI TMP, Zero
jmp sec2
sec2:
    MOV LED2, TMP
    MOV TMP, LED3

```

<...>

```

end_tim1:
POP TMP
OUT SREG, TMP ; Восстановление значения SREG из стека
POP TMP ; Восстановление значения TMP из стека
RETI ; Возврат из обработчика прерываний и разрешение прерываний

```

```

;-----
-----

```

EEPROM_write:

```

; Wait for completion of previous write
sbic EECR, EWE
rjmp EEPROM_write
; Set up address (r18:r17) in address register
out EEARH, R18
out EEARL, R17
; Write data (r16) to data register
out EEDR, ZET
; Write logical one to EEMWE
sbi EECR, EEMWE
; Start eeprom write by setting EWE
sbi EECR, EWE
ret

```

EEPROM_read:

```

; Wait for completion of previous write
sbic EECR, EWE

```

```

rjmp EEPROM_read
; Set up address (r18:r17) in address register
out EEARH, R18
out EEARL, R17
; Start eeprom read by writing EERE
sbi EECR,EERE
; Read data from data register
in ZET,EEDR
ret

```

5. Ответы на контрольные вопросы

1) Посредством каких регистров производится конфигурирование таймера-счётчика?

- WCM00, WCM01 - настраивают режим работы;
- CS02, CS01, CS00 - настройка тактовой частоты;
- CM01, CM00 - поведение при выводе OC0;
- TCNT0 - счетный регистр таймера T0;
- OCR0 - регистр сравнения;

2) Какие источники импульсов могут применяться для увеличения таймера-счётчика и для каких целей?

Таймер/Счетчик (далее буду звать его Т/С) считает либо тактовые импульсы от встроенного тактового генератора, либо со счетного входа.

3) В каких режимах могут работать таймеры-счётчики?

WCM00	WCM01	режим
0	0	normal
0	1	PWM
1	0	CTC - сброс
1	1	Fast PWM

4) Как рассчитать начальное значение таймера-счётчика по заданному времени, которое должен отмерить таймер-счётчик до своего переполнения?

$8000000 \text{ Гц} / \text{Тактовая частота} / \text{начальное значения ТС} = \text{кол-во раз срабатывания в секунду}$, следовательно:

$\text{начальное значения ТС} = 8000000 \text{ Гц} / \text{Тактовая частота} / \text{кол-во раз срабатывания в секунду}$

5) В чём состоит отличие работы таймера-счётчика в режиме таймера и в режиме счётчика?

Таймеры общего назначения используются для формирования различных интервалов времени и прямоугольных импульсов заданной частоты. Кроме того, они могут работать в режиме счетчика и подсчитывать тактовые импульсы

заданной частоты, измеряя таким образом длительность внешних сигналов, а также при необходимости подсчитывать количество любых внешних импульсов.